Generalizing Reinforcement Learning-Based Quantum Circuit Synthesis across Multiple Topologies

Giovanni Acampora[1], Allegra Cuzzocrea[1], Marco Lapegna[2], Roberto Schiattarella[1], Autilia Vitiello[1]
[1] *Department of Physics "Ettore Pancini"*, University of Naples Federico II, Naples, Italy
[2] *Department of Mathematics and Applications "Renato Caccioppoli"*, University of Naples Federico II, Naples, Italy
{giovanni.acampora, allegra.cuzzocrea, marco.lapegna, roberto.schiattarella, autilia.vitiello}@unina.it

***Abstract*—The optimization of the limited resources of current quantum devices hinges on the implementation of quantum-compiling techniques. The process of transforming high-level quantum algorithms into executable quantum circuits has a significant impact on the performance and fidelity of computations. One of the most critical aspects of the compiling stack is the synthesis of quantum circuits, which aims to minimize the depth and number of gates, as well as the error rate. Nevertheless, the optimal synthesis algorithms are extremely computationally expensive. This is why synthesis algorithms based on reinforcement learning (RL) have recently been introduced. However, such algorithms are constrained by the topology of quantum processors, necessitating retraining of the RL agent for different topologies. To achieve a balance between adaptability, optimality, and computational cost in RL-based quantum circuit synthesis techniques, this work proposes an RL-based quantum circuit synthesis algorithm of Clifford operators that addresses different topologies simultaneously. Experimental tests demonstrate that our synthesizer outperforms state-of-the-art greedy techniques while being significantly faster than boolean satisfiability solvers (SAT).**

***Index Terms*—Quantum Circuit Synthesis, Reinforcement Learning, Quantum Compiling, Quantum Computing**

I. Introduction

The promise of quantum computers is to execute algorithms that solve problems for which classical computers require an excessive amount of resources [1], [2], [3]. However, there is still a big gap between the current status of quantum computers and the promise that quantum computation offers in fields such as chemistry [4] or artificial intelligence [5]. Indeed, the current Noisy Intermediate Scale Quantum (NISQ) devices are characterized by a limited number of qubits with a high level of noise which strongly affects the reliability of the computation [6]. Moreover, each NISQ device has its own characteristics and to execute a quantum algorithm, this must be adapted to the specific device through a quantum compiling process [7] which introduces further noise in the computation. To bridge the gap towards an actual quantum advantage three paths are currently under investigation: design better algorithms that require less hardware resources, build better quantum hardware, and optimize the compiling procedure. Quantum compiling, also known as *transpiling*, involves several steps that make an abstract quantum algorithm something that can be physically executed on a given quantum processor. Among these steps, there is quantum circuit synthesis, which aims to construct a quantum circuit that implements a given quantum operator using a given set of quantum gates that can be used on the hardware. A good synthesizer must reach a twofold goal: optimize the number of gates required to compute the given quantum operator in order to reduce the noise affecting the computation, and at the same time make the quantum circuit implementing the operator compatible with the topology and the basis gate set of a given hardware. Recently, in [8] a quantum circuit synthesis algorithm based on reinforcement learning (RL) has been proposed. In detail, an agent learns how to synthesize quantum circuits through directed trial and error. The agent selects a gate within a gate set and adds it to a quantum circuit in order to process it and provide the new state and a reward. The agent succeeds when the constructed circuit implements the target operator. However, this approach only synthesizes Clifford quantum circuits and it can handle only a specific quantum processor topology at a time. This means that an RL agent must be retrained every time a different quantum processor coupling map is considered. To overcome this limitation, the proposed research extends the work in [8] by exploiting a generic approach for Clifford synthesis over an arbitrary set of quantum processor topologies. In this way, for a fixed number of qubits, it is possible to obtain an RL model that allows for a near-optimal synthesis of Clifford circuits and their automatic adaptation to a given architecture in a short amount of time. This goal is reached by exploiting the invalid action masking technique, i.e. the RL agent is automatically forced to perform actions according to the topology of the processor on which the quantum circuit must be synthesized. Moreover, this research explicitly states a reward function that can be used as a benchmark for further studies on RL-based algorithms for quantum circuit synthesis. The experimental studies reported in this paper show that the proposed methodology strongly outperforms Qiskit SDK's greedy synthesis algorithms [9], [10] and it is orders of magnitude faster than SAT solvers. Overall, the introduced methodology aims to balance the adaptability, optimality, and computational cost of RL-based

This work was supported by the National Center for HPC, Big Data, and Quantum Computing (Spoke 1: Future HPC) in the framework of Next Generation EU program.

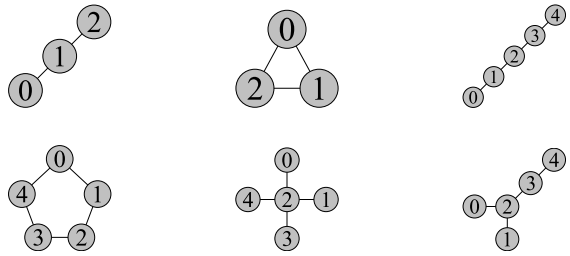


Fig. 1: 3-qubit and 5-qubit topologies used during the training.

quantum circuit synthesis techniques.

The code related to this paper is available at the following link: https://github.com/Quasar-UniNA/Topology_Aware_Quantum_Circuit_Synthesis_with_RL.git.

II. RELATED WORK

In recent years, artificial intelligence has been widely explored to implement efficient quantum compilers [11], [12]. One extensively researched area is quantum circuit synthesis. Initially, metaheuristics were proposed to address this task, as in [13], where a genetic algorithm was exploited to perform the synthesis though without considering the connectivity constraints. Differently, in [14] a topology-aware synthesis process based on the A* search strategy was introduced. However, this approach was extremely slow for practical use in actual quantum compilers. To speed up the synthesis procedure the original A* search was extended in [15], [16]. More recently, researchers have started to investigate machine learning (ML) to reduce the exponentially growing run times of previous algorithms [17]. However, classical ML algorithms as deep neural networks still suffer from scalability issues [18]. To overcome that, RL-based synthesizers have emerged as a promising tool. Initially, works as [19], [20], [21] have investigated the feasibility of RL-based synthesizers focusing on single-qubit quantum state preparation and gate compilation. An attempt to extend the synthesis to two-qubit unitaries was proposed in [22] by combining deep Q-learning and A* search. However, the search complexity still grows exponentially. Alternative RL-based approaches were introduced in [23], [24], [25]; however, none of these synthesizers account for the topology constraints of quantum processors. Differently, in [26] a deep-RL strategy to optimize local quantum circuits on arbitrary topologies was proposed, while in [27] it was introduced a quantum compiler for a specific superconducting processor obtaining satisfactory results for two-qubit and three-qubit systems. If the above researches focus on the optimization of generic quantum circuits, many scientists are focusing on the synthesis of just Clifford circuits. This class of circuits is very peculiar because they can be simulated efficiently on classical computers [10]. The synthesis of this class of operators has been addressed with greedy approaches in [10], [28], [29], [9] still not taking into account the topology of different quantum processors or considering just linear architectures. Very recently, in [8], the RL-based Clifford synthesis strategy was introduced. Despite this technique having surpassed the effectiveness of the Qiskit SDK's greedy synthesis algorithm, it has the limitation that the RL agent must be trained on a specific processor topology. Considering the above literature analysis, the proposed work is the first to introduce a Clifford synthesis approach based on RL which is topology-aware.

III. BACKGROUND

This section aims to introduce the principal aspects of quantum computing and quantum transpiling, focusing on the synthesis problem.

A. Quantum Computing and Clifford Circuits

Quantum computing takes advantage of the principles of quantum mechanics to process information in an innovative way. While the fundamental unit of information in classical computation is the bit, quantum computers work with the so-called *qubits*. Unlike a classical bit, which can be either 0 or 1, a qubit can exist in a state of 0, 1, or any quantum superposition of these states. Formally, a quantum state describing a qubit is represented as follows:

$$|\phi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{1}$$

where α and β are two complex numbers such that $|\alpha|^2 + |\beta|^2 = 1$ and their square modulus represents the probability of reading the qubit in the state 0 or 1 respectively. Similarly, a quantum state describing a system of N qubits is defined as:

$$|\psi\rangle = \sum_{i=0}^{2^N-1} \alpha_i\,|i\rangle \tag{2}$$

where $\alpha_i \in \mathcal{C}$ and as in the single-qubit case, it represents the probability of reading the classical state i. Here, i denotes the integer representation of the binary string composed of N bits. In quantum computing, quantum states are manipulated by quantum gates. A quantum circuit is a sequence of quantum gates and represents the structure through which quantum algorithms are implemented. Clifford circuits represent a particular class of quantum circuits. These circuits are composed of gates that belong to the Clifford group (H, S, CX)[10]. A Clifford circuit, i.e. a quantum circuit composed of Clifford gates, is also known as a stabilizer circuit. According to Gottesman-Knill theorem [30], a stabilizer circuit can be simulated in polynomial time on classical probabilistic computers. As introduced in [10], an N-qubit stabilizer state, i.e. an N-qubit quantum state obtained by a stabilizer circuit, can be described through a tableau of binary variables. This is a $2N \times 2N$ binary matrix plus a $2N$ boolean phase vector. Once a new Clifford gate is applied to it, the tableau can be updated in *O(N)*, according to the transformation rules introduced in [10].

B. Quantum Transpiling and Synthesis

Quantum transpiling is the process of converting a high-level algorithm into a lower-level quantum circuit that can be executed on a specific quantum hardware platform. In fact, current NISQ devices have their native gate set and their specific connectivity constraints, that depend on the processor

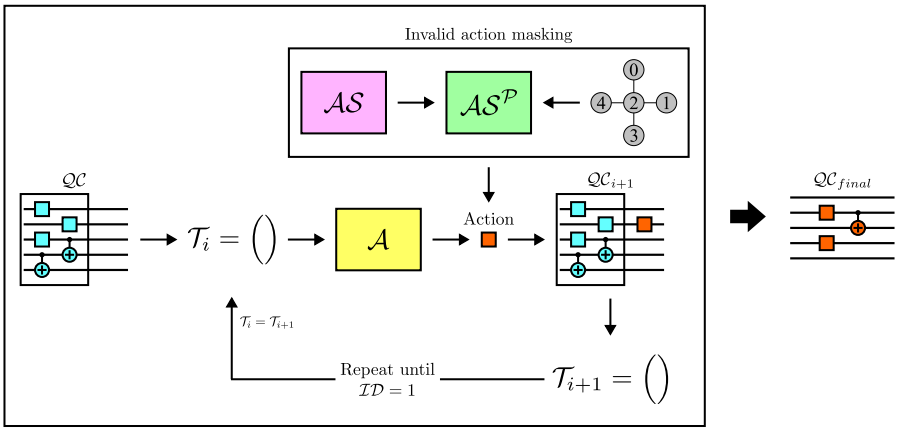


Fig. 2: Process of the proposed RL-based quantum circuit synthesis approach.

topology. The presence of a native gate set implies that only a subset of quantum gates can be physically executed on a device, thereby a generic quantum circuit must be translated into such a gate set. On the other hand, to minimize the effect of quantum noise, the quantum processors are built with a given topology. Fig. 1 reports some examples related to topologies made up of three and five qubits. This means that quantum gates involving two qubits can be physically executed only if the two qubits are coupled in the topology. In a quantum transpiler, two operations are carried out to adapt the quantum circuits to a given processor topology. Firstly, the 'logical' qubits in a quantum circuit are mapped on the physical qubits of the quantum processor. Secondly, SWAP gates are inserted to move the state between qubits [11]. This procedure is commonly known as the quantum circuit routing. However, considering that quantum gates, particularly two-qubit gates like the CX, are extremely noisy objects and that a single SWAP gate is implemented through three CXs, moving around the state of the qubits is a very noisy operation. An effective transpiler indeed is a transpiler that minimizes the number of gates to execute for implementing a given quantum circuit, focusing specifically on minimizing the number of two-qubit gates. This minimization can be reached in two ways: first, by exploiting a good mapping and routing strategy; and second, by using a good quantum circuit synthesis technique. This last step is crucial in the transpiling process and consists of optimizing the quantum circuit that implements a given algorithm or operator. Formally, given a quantum operator U and a set of gates $\mathcal{S}$, the synthesis consists of making up a quantum circuit with the gates in $\mathcal{S}$ and that implements the operator U. A good synthesizer is capable of producing optimal or near-optimal circuits with a reasonable amount of computational resources.

IV. Circuit synthesis with reinforcement learning

The problem of synthesizing Clifford circuits can be reformulated as a sequential decision process, as stated in [8]. To describe this process, let us synthesize a target N-qubits Clifford operator U that is originated from a Clifford circuit $\mathcal{QC}$. The description of U as its Tableau $\mathcal{T}$ represents the initial state of the decision problem. Moreover, let us define with $\mathcal{A}$ the RL-agent, with $\mathcal{P}$ the topology of the N-qubits quantum processor on which U must be synthesized, and with $\mathcal{AS}$ the action space of $\mathcal{A}$. $\mathcal{AS}$ is a discrete action space composed of all the basis Clifford gates that can be applied on the N-qubits. Precisely, $\mathcal{AS} = \{H_i, S_i, CX_{ij}\}$, where $i, j \in [1, N]$, $i \neq j$, H_i and S_i respectively represent a Hadamard and a Phase gate applied on the i-th qubit, while CX_{ij} is a CX gate where the i-th qubit is the control and the j-th qubit is the target. Thanks to the invalid action masking technique introduced in this research, given a topology $\mathcal{P}$, the action space from which $\mathcal{A}$ will select its action will be a subspace $\mathcal{AS}^{\mathcal{P}} = \{H_i, S_i, CX_{ij}^{\mathcal{P}}\} \subset \mathcal{AS}$ composed of all the single-qubit gates in $\mathcal{AS}$ and only the CX_{ij} gates that respect the coupling map of $\mathcal{P}$, here denoted as $CX_{ij}^{\mathcal{P}}$. Considering this, the agent $\mathcal{A}$ is trained in such a way that it will be able to select from $\mathcal{AS}^{\mathcal{P}}$ the sequence of gates that will result in a quantum circuit implementing the desired Clifford operator U and that is compliant with the coupling map of $\mathcal{P}$. The process is sequential because once $\mathcal{A}$ selects a gate from $\mathcal{AS}^{\mathcal{P}}$, and this is applied to $\mathcal{QC}$ the new Tableau $\mathcal{T}_1$ is computed according to the composition rules in [10] and $\mathcal{A}$ will select a new action according to this new observation space. This process is iterated until the $2N \times 2N$ matrix describing the final Tableau is equal to the identity, which means that the original Clifford operator U has been successfully synthesized, except for a phase factor. The synthesized quantum circuit will consist of all the gates added from $\mathcal{A}$ to the original circuit $\mathcal{QC}$. Fig. 2 graphically summarizes this process. It is worth noting that since all the gates inserted in the synthesized quantum circuits come from $\mathcal{AS}^{\mathcal{P}}$, such a circuit does not need a further routing step. In the next section, details of how the agent $\mathcal{A}$ was trained will be provided.

A. Training Procedure

The RL algorithm exploited in this research is the Proximal Policy Optimization (PPO) algorithm [31] equipped with

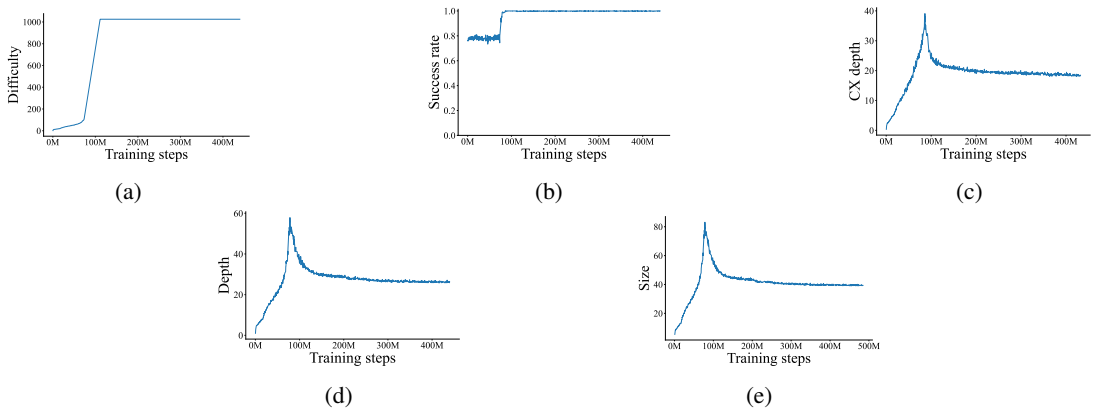


Fig. 3: Training progress related to the training of the MLP policy for 5-qubit Clifford quantum circuits. The evolution of the quantities of interest throughout the training steps is reported. In detail, Fig. (a) reports the training difficulty, Fig. (b) the success rate, Fig. (c) the CX depth, Fig. (d) the depth, and Fig. (e) the total number of gates.

an invalid action masking technique [32]. PPO is an on-policy algorithm that samples data by interacting with the environment and optimizes a surrogate objective function through stochastic gradient ascent. In detail, the policy is updated after a rollout, i.e. the process of interacting with the environment using the current policy to generate a sequence of experiences. These experiences include state-action-reward sequences that are used to update the policy. To speed up the rollout data collection, a vectorized environment can be used, i.e. a given number of environment copies can be generated, and the agent collects data simultaneously on them. Thanks to the invalid action masking technique, the probability distribution of actions outputted by the agent is modified by replacing the unwanted scores with a large negative number. This mechanism enables the training phase to be executed only once for N-qubit Clifford operators and a given set of topologies. Indeed, the invalid action masking technique selects the correct action subspace $\mathcal{AS}^{\mathcal{P}}$ according to the topology of the processor $\mathcal{P}$ to use. The policy is trained using random Clifford operators by exploiting a curriculum learning strategy as in [8]. This strategy dynamically increases a difficulty parameter d regarding the input Clifford operators to synthesize. In the first phase of our training procedure, this difficulty corresponds to the number of gates composing the random Clifford operator. Random Clifford operators are generated from N-qubit quantum circuits made up of d gates uniformly sampled from $\mathcal{AS}^{\mathcal{P}}$, where for each different operator, $\mathcal{P}$ is also selected randomly from a set of available topologies. The parameter d is initially set to 1 and it is incremented by one at the end of each policy update if the success rate, i.e. the ratio between the number of successful episodes and the total number of episodes in a rollout, is greater or equal to a given threshold. Here, an episode is a different random Clifford operator to be synthesized, and each episode ends when the operator is successfully synthesized or when a maximum number of actions (steps) is reached. The maximum number of steps for an episode is set to d, in such a way as to minimize the depth of the circuit resulting from the synthesis. In the second phase of training, once the difficulty is above a certain level, the target operators provided as input are generated differently. Specifically, random Clifford operators are sampled directly from the N-qubit dimensional Clifford space and not anymore from the quantum circuits obtainable by adding d gates from $\mathcal{AS}^{\mathcal{P}}$. This aims to achieve success in synthesizing any Clifford operator for a given number of qubits.

As for the reward function used in our experiments, this was properly designed. Eq. 3 reports its explicit form both in the case of a successful action, i.e. an action of $\mathcal{A}$ which results in the correct synthesis of the Clifford operator, and in the case of not a successful action.

$$\mathcal{R} = \begin{cases} 1000 - \\ D_i(D_i - D_{i-1}) - c_0 Tq_i & \text{if Success} \\ \\ \mathcal{ID} - c_1(D_i - D_{i-1}) - \\ c_2(Tq_i - Tq_{i-1}) - & \text{if not Success} \\ c_3(Sq_i - Sq_{i-1}) \end{cases} \tag{3}$$

D_i represents the depth of the synthesized quantum circuit after the i-th action of $\mathcal{A}$; Tq_i and Sq_i are the numbers of two-qubit and single-qubit gates, respectively; c_0, c_1, c_2, and c_3 are coefficients; $\mathcal{ID}$ is the inverse distance from the $2N \times 2N$ identity matrix of the $2N \times 2N$ matrix M describing $\mathcal{T}_i$ except for the phase factor, after the i-th action of $\mathcal{A}$. $\mathcal{ID}$ is computed as:

$$\mathcal{ID} = 1 - \frac{\sum_{lm} I_{lm} - M_{lm}}{2N \times 2N} \tag{4}$$

where I_{lm} and M_{lm} are the elements on the l-row and the m-column of the identity matrix and the matrix describing the Tableau, respectively. Therefore, $\mathcal{ID} \in [0, 1]$ and the greater the distance of the matrix $\mathcal{T}_i$ from the identity one, the lower the $\mathcal{R}$ value. Specifically, the reward is designed to give a great positive value if $\mathcal{A}$ achieves success. This value will be higher, the lower the depth and the number of 2-qubit gates in

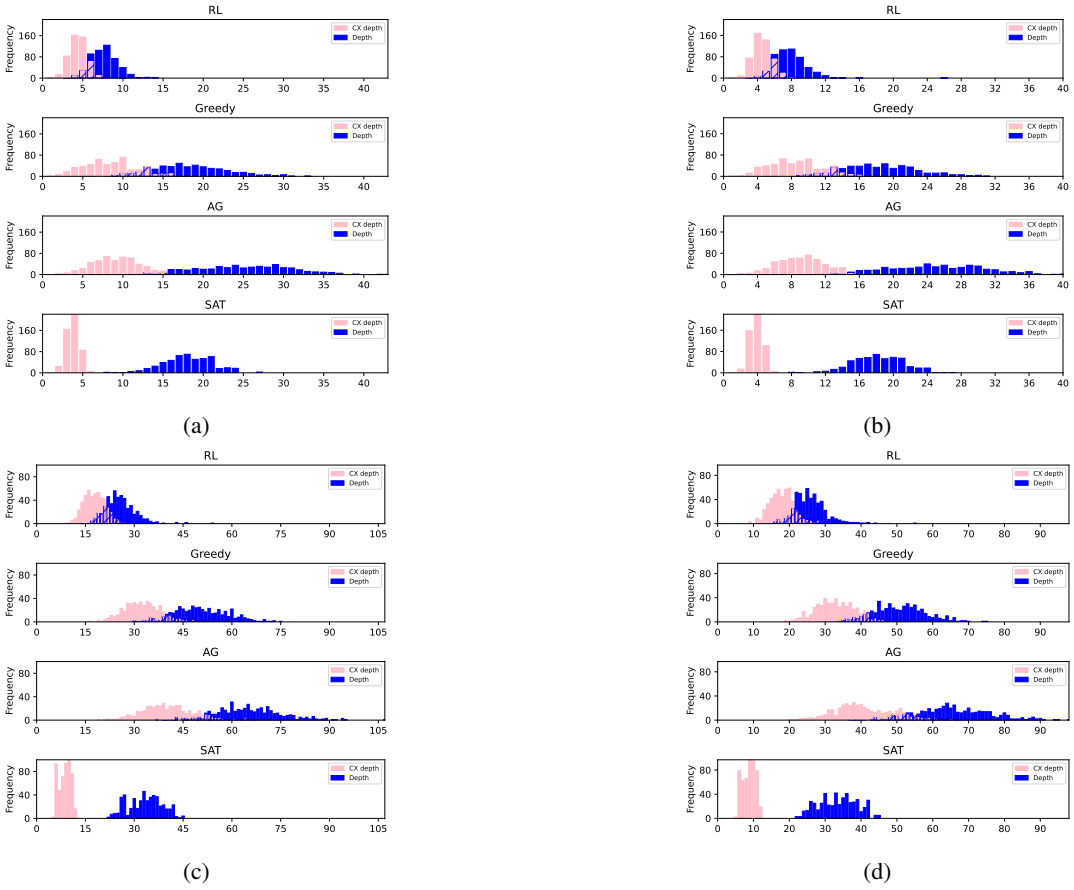


Fig. 4: Comparison between CX depth and depth values of the circuits resulting from the 3-qubit and 5-qubit models and those resulting from the greedy approaches and the SAT solver. Both routed input circuits (a) and (c) and non-routed input circuits (b) and (d) scenarios are reported.

the synthesized circuit. If the action of $\mathcal{A}$ does not conclude the synthesis process, $\mathcal{R}$ will be a value between 0 and 1, which will tend to 0 the further the tableau resulting from the action is from the identity, penalizing actions that add depth, single-qubit or two-qubit gates to the synthesized circuit according to their coefficients.

V. Experiments and Results

The experimental session involves the training and testing of models capable of synthesizing 3-qubit and 5-qubit Clifford operators. In this section, the training procedure introduced in Sec. IV-A will be detailed, and it will be followed by an evaluation of the performance of the trained models. This will be accomplished by comparing our models with state-of-the-art synthesizers, namely [1]SAT solver (SAT), the Qiskit SDK's greedy synthesis algorithm [9] (Greedy), and the Qiskit SDK's synthesis algorithm based on Aaronson-Gottesman method [10] (AG). The last two algorithms are full connectivity synthesis methods, so they are followed by routing with Qiskit SDK's SABRE [33].

[1]https://github.com/qiskit-community/qiskit-sat-synthesis

A. Training Setup

In our experiments, the policy of $\mathcal{A}$ was set to be a multi-layer perceptron (MLP). To speed up the training process 40 parallel environments were built to collect data simultaneously. They were created through the use of the Gymnasium interface [34], while the MLP default policy in the MaskablePPO class implemented in Stable Baselines3 [35] was exploited for the policy. During the training of the related models, all possible processor topologies of 3 and 5 qubits were considered. They are graphically reported in Fig. 1. These topologies can be sub-topologies of larger devices, meaning that the models trained on them can be useful also to synthesize and map Clifford circuits on larger systems. As for the threshold for increasing the difficulty level d, a value of success rate of 0.8 was set. This means that the difficulty of the operators to be synthesized during the training was increased if the 80% of the operators present in a rollout data collection were successfully synthesized. The second training phase, which consists of using random Cliffords from the Clifford space, started once the difficulty level reached 1024. These operators were generated using the method described in [29]. After a large experimentation, we set the parameters in the reward

function as follows: $c_0 = 10$, $c_1 = 0.5$, $c_2 = 0.15$, $c_3 = 0.1$. As an example, Fig. 3 shows the progress related to the training of the MLP policy for 5-qubit Clifford quantum circuits. As can be seen, during a very short initial phase, the policy very quickly learns how to synthesize the Clifford operators with difficulty 1. Then, the success rate remains around the threshold while the difficulty starts incrementing, meaning that the model is progressively learning to synthesize harder operators. Finally, the success rate stabilizes at 1 implying that the model can synthesize 100% of input Cliffords. In this phase of training, the difficulty remains to 1025 meaning that the input Clifford operators are sampled directly from the Clifford space. Fig. 3 also shows the development of the CX depth and depth of the synthesized circuits. They initially increase up to a certain value and when the success rate is stable around 1, they start decreasing implying an improvement in performance.

B. Model Testing

In this section, the testing procedure of the two models trained as discussed in Sec. V-A is presented. For both the 3-qubit and 5-qubit models, two different testing scenarios have been considered. In the first scenario, the models were tested on random Clifford operators related to quantum circuits composed exclusively of H, S, and CX gates that already respected the topology constraints. In the second scenario, the random Clifford operators considered were directly sampled from the Clifford space and they were not related to particular quantum circuits that respect any topological constraints or have limitations on the gates used. This second step is crucial to evaluate how our models can synthesize generic Clifford operators on each processor topology considered. In both scenarios, 500 3-qubit and 5-qubit Clifford operators were synthesized evenly divided over the topologies in Fig. 1. Remarkably, for both the 3-qubit and 5-qubit cases, the trained models were able to synthesize 100% of the test operators. This result is very important for the second set of test operators because it indicates that our models can synthesize a completely random Clifford operator on any topology with no constraints on the gate set used to generate it. Fig. 4 reports the comparison with the other synthesis approaches. It is important to highlight that the Greedy and the AG approaches do not take into account the topology of the processor during the synthesis, therefore the quantum circuit generated by them is routed using the SABRE algorithm. This step is not required for the SAT and our RL approach, which generate quantum circuits already compliant with a given processor topology. The metrics used for our comparison are the CX depth and depth of the synthesized quantum circuits. The lower these two values, the lower the number of 1-qubit and 2-qubit gates in the circuit and therefore the lower the effect of quantum noise affecting the computation. Figs. 4a and 4c report the distributions of these values obtained on the Clifford operators related to topology complainant input quantum circuits for 3 and 5 qubits respectively, while Figs. 4b and 4d report the same distributions for the Clifford operators sampled directly from the Clifford space. As can be seen, in both input scenarios

		Synthesis Algorithm			
Test Scenario	Number of Qubits	RL	Greedy	AG	SAT
1	3	0.075	0.010	0.004	1.548
	5	0.295	0.018	0.006	14.96
2	3	0.080	0.009	0.004	1.606
	5	0.298	0.017	0.006	15.24

TABLE I: Average execution times in seconds of the tested synthesis methodologies.

Number of Qubits	PMH	KMS
3	15%	59%
5	36%	60%

TABLE II: Average execution times in seconds for PMH and KMS synthesis methods.

the proposed synthesis methodology presents smaller values for CX depth and depth with respect to the Greedy and AG approaches. In particular, it reduces the CX depth average by 48% and 50% compared to Greedy and AG, respectively, in the 3-qubit case, and by 45% and 56% in the 5-qubit case. Besides, it shows a reduction in depth average by 58%, 70%, and 58% compared to Greedy, AG, and SAT, respectively, in the 3-qubit case, and by 49%, 60%, and 23% in the 5-qubit case. Despite the performance in terms of depth surpassing also that of the SAT solver, as expected, the SAT solver demonstrates better performance in terms of CX depth. This is because it gives optimal results in terms of two-qubit gate depth. However, SAT is an extremely slow algorithm, and this limits its applicability in actual quantum transpiler systems. Indeed, as reported in Tab. II it is orders of magnitude slower than the proposed RL approach.

VI. Conclusion

This paper extends the previously introduced RL-based Clifford circuit synthesis mechanism [8]. In detail, it solves one of the most important criticalities of the previous approach, i.e. the necessity of training different RL agents to perform synthesis on different quantum processor topologies. Indeed, training such an RL agent is a procedure that requires a large amount of time and computational resources. This goal is reached by exploiting an invalid action masking schema in the training of a PPO algorithm, which results in an agent capable of mapping a Clifford operator on any processor topology seen during the training. As shown from the experimental tests carried out considering a thousand of 3-qubit and 5-qubit Clifford operators, the proposed approach achieves better results compared to the state-of-the-art greedy techniques both in terms of depth and CX depth of the synthesized quantum circuits. Moreover, its performance is comparable with those of an exact solver, but with a synthesis runtime advantage of several orders of magnitude. In the future, the RL-based synthesizer proposed in this research will be extended to Clifford circuits composed of a greater number of qubits and, consequently, to more topologies. Furthermore, particular attention will be given to extending the approach to non-Clifford circuits.

References

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2011. [Online]. Available: https://www.amazon.com/Quantum-Computation-Information-10th-Anniversary/dp/1107002176?SubscriptionId=AKIAIOBINVZYXZQZ2U3A&tag=chimbori05-20&linkCode=xm2&camp=2025&creative=165953&creativeASIN=1107002176

[2] F. Arute et al., "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, no. 7779, pp. 505–510, Oct. 2019. [Online]. Available: https://doi.org/10.1038/s41586-019-1666-5

[3] N. Herrmann, D. Arya, M. W. Doherty, A. Mingare, J. C. Pillay, F. Preis, and S. Prestel, "Quantum utility – definition and assessment of a practical quantum advantage," in *2023 IEEE International Conference on Quantum Software (QSW)*, 2023, pp. 162–174.

[4] Y. Cao, J. Romero, J. P. Olson, M. Degroote, P. D. Johnson, M. Kieferová, I. D. Kivlichan, T. Menke, B. Peropadre, N. P. D. Sawaya, S. Sim, L. Veis, and A. Aspuru-Guzik, "Quantum chemistry in the age of quantum computing," *Chemical Reviews*, vol. 119, no. 19, pp. 10 856–10 915, 2019, pMID: 31469277. [Online]. Available: https://doi.org/10.1021/acs.chemrev.8b00803

[5] H.-Y. Huang, M. Broughton, M. Mohseni, R. Babbush, S. Boixo, H. Neven, and J. R. McClean, "Power of data in quantum machine learning," *Nature Communications*, vol. 12, no. 1, p. 2631, May 2021. [Online]. Available: https://doi.org/10.1038/s41467-021-22539-9

[6] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, Aug. 2018. [Online]. Available: https://doi.org/10.22331/q-2018-08-06-79

[7] M. Maronese, L. Moro, L. Rocutto, and E. Prati, *Quantum Compiling*. Cham: Springer International Publishing, 2022, pp. 39–74. [Online]. Available: https://doi.org/10.1007/978-3-030-89746-8_2

[8] D. Kremer, V. Villar, H. Paik, I. Duran, I. Faro, and J. Cruz-Benito, "Practical and efficient quantum circuit synthesis and transpiling with reinforcement learning," *arXiv preprint arXiv:2405.13196*, 2024.

[9] S. Bravyi, R. Shaydulin, S. Hu, and D. Maslov, "Clifford Circuit Optimization with Templates and Symbolic Pauli Gates," *Quantum*, vol. 5, p. 580, Nov. 2021. [Online]. Available: https://doi.org/10.22331/q-2021-11-16-580

[10] S. Aaronson and D. Gottesman, "Improved simulation of stabilizer circuits," *Phys. Rev. A*, vol. 70, p. 052328, Nov 2004. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevA.70.052328

[11] G. Acampora and R. Schiattarella, "Deep neural networks for quantum circuit mapping," *Neural Computing and Applications*, vol. 33, no. 20, pp. 13 723–13 743, 2021.

[12] J. Kusyk, S. M. Saeed, and M. U. Uyar, "Survey on quantum circuit compilation for noisy intermediate-scale quantum computers: Artificial intelligence to heuristics," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–16, 2021.

[13] B. Rubinstein, "Evolving quantum circuits using genetic programming," in *Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No.01TH8546)*, vol. 1, 2001, pp. 144–151 vol. 1.

[14] M. G. Davis, E. Smith, A. Tudor, K. Sen, I. Siddiqi, and C. Iancu, "Towards optimal topology aware quantum circuit synthesis," in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2020, pp. 223–234.

[15] E. Smith, M. G. Davis, J. Larson, E. Younis, L. B. Oftelie, W. Lavrijsen, and C. Iancu, "Leap: Scaling numerical optimization based synthesis using an incremental approach," *ACM Transactions on Quantum Computing*, vol. 4, no. 1, feb 2023. [Online]. Available: https://doi.org/10.1145/3548693

[16] M. Weiden, J. Kalloor, J. Kubiatowicz, E. Younis, and C. Iancu, "Wide quantum circuit optimization with topology aware synthesis," in *2022 IEEE/ACM Third International Workshop on Quantum Computing Software (QCS)*, 2022, pp. 1–11.

[17] M. Weiden, E. Younis, J. Kalloor, J. Kubiatowicz, and C. Iancu, "Improving quantum circuit synthesis with machine learning," in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Los Alamitos, CA, USA: IEEE Computer Society, sep 2023, pp. 1–11. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/QCE57702.2023.00093

[18] K. Murakami and J. Zhao, "Automated synthesis of quantum circuits using neural network," in *Proceedings - 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security, QRS 2022*, ser. IEEE International Conference on Software Quality, Reliability and Security, QRS. United States: Institute of Electrical and Electronics Engineers Inc., 2022, pp. 694–702, publisher Copyright: © 2022 IEEE.; 22nd IEEE International Conference on Software Quality, Reliability and Security, QRS 2022 ; Conference date: 05-12-2022 Through 09-12-2022.

[19] M. S. Alam, N. F. Berthusen, and P. P. Orth, "Quantum logic gate synthesis as a Markov decision process," *npj Quantum Information*, vol. 9, no. 1, p. 108, Oct. 2023. [Online]. Available: https://doi.org/10.1038/s41534-023-00766-w

[20] L. Moro, M. G. A. Paris, M. Restelli, and E. Prati, "Quantum compiling by deep reinforcement learning," *Communications Physics*, vol. 4, no. 1, p. 178, Aug. 2021. [Online]. Available: https://doi.org/10.1038/s42005-021-00684-3

[21] Y.-H. Zhang, P.-L. Zheng, Y. Zhang, and D.-L. Deng, "Topological quantum compiling with reinforcement learning," *Phys. Rev. Lett.*, vol. 125, p. 170501, Oct 2020. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.125.170501

[22] Q. Chen, Y. Du, Y. Jiao, X. Lu, X. Wu, and Q. Zhao, "Efficient and practical quantum compiler towards multi-qubit systems with deep reinforcement learning*," *Quantum Science and Technology*, vol. 9, no. 4, p. 045002, jul 2024. [Online]. Available: https://dx.doi.org/10.1088/2058-9565/ad420a

[23] T. Gabor, M. Zorn, and C. Linnhoff-Popien, "The applicability of reinforcement learning for the automatic generation of state preparation circuits," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, ser. GECCO '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 2196–2204. [Online]. Available: https://doi.org/10.1145/3520304.3534039

[24] M. Kölle, T. Schubert, P. Altmann, M. Zorn, J. Stein, and C. Linnhoff-Popien, "A reinforcement learning environment for directed quantum circuit synthesis," 2024. [Online]. Available: https://arxiv.org/abs/2401.07054

[25] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer, "Unitary synthesis of clifford+t circuits with reinforcement learning," 2024. [Online]. Available: https://arxiv.org/abs/2404.14865

[26] T. Fösel, M. Y. Niu, F. Marquardt, and L. Li, "Quantum circuit optimization with deep reinforcement learning," 2021. [Online]. Available: https://arxiv.org/abs/2103.07585

[27] Z. T. Wang, Q. Chen, Y. Du, Z. H. Yang, X. Cai, K. Huang, J. Zhang, K. Xu, J. Du, Y. Li, Y. Jiao, X. Wu, W. Liu, X. Lu, H. Xu, Y. Jin, R. Wang, H. Yu, and S. P. Zhao, "Quantum compiling with reinforcement learning on a superconducting processor," 2024. [Online]. Available: https://arxiv.org/abs/2406.12195

[28] D. L. Maslov and M. Roetteler, "Shorter stabilizer circuits via bruhat decomposition and quantum circuit transformations," *IEEE Transactions on Information Theory*, vol. 64, pp. 4729–4738, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:22092011

[29] S. Bravyi and D. Maslov, "Hadamard-free circuits expose the structure of the clifford group," *IEEE Transactions on Information Theory*, vol. 67, no. 7, p. 4546–4563, Jul. 2021. [Online]. Available: http://dx.doi.org/10.1109/TIT.2021.3081415

[30] D. Gottesman, "The heisenberg representation of quantum computers," 1998. [Online]. Available: https://arxiv.org/abs/quant-ph/9807006

[31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[32] W. Ding, Z. Ming, G. Wang, and Y. Yan, "System-of-systems approach to spatio-temporal crowdsourcing design using improved ppo algorithm based on an invalid action masking," *Knowledge-Based Systems*, vol. 285, p. 111381, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950705124000169

[33] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for nisq-era quantum devices," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1001–1014. [Online]. Available: https://doi.org/10.1145/3297858.3304023

[34] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," 2016. [Online]. Available: https://arxiv.org/abs/1606.01540

[35] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning

implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html